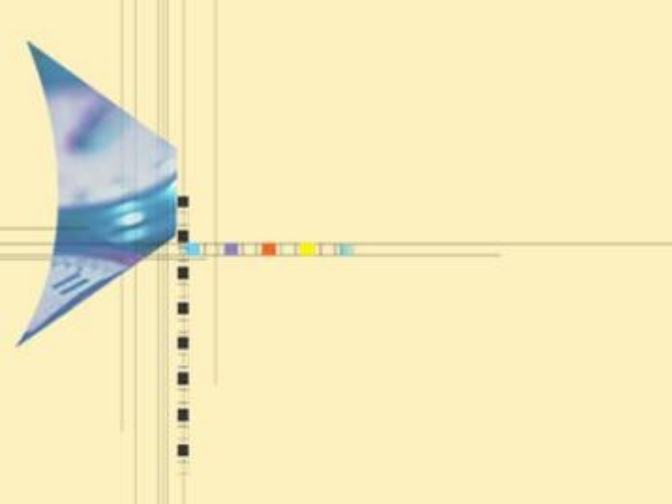


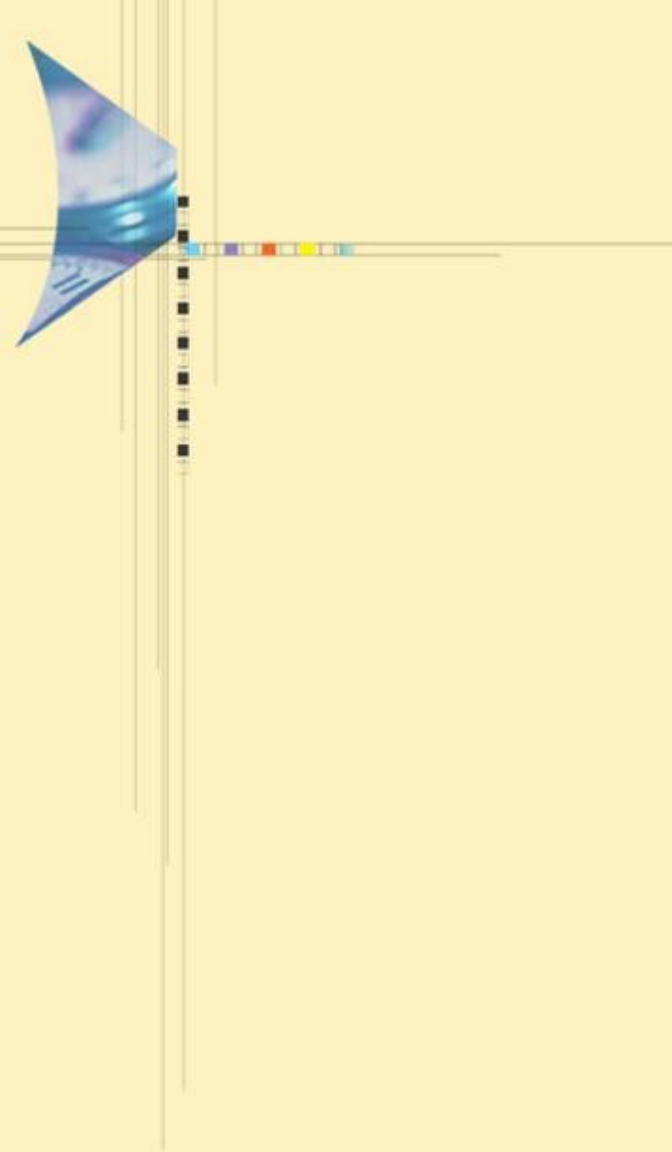


算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

周一(3-4，三区1-502)，周三（6-7，三区1-325）



第二讲

归纳法

主要内容



2.1 引言

2.2 归纳算法的简单例

2.3 归纳算法的设计思想

2.4. 归纳策略的经典应用



递归技术

递归：将一个问题分割成一个或多个的子问题，这些子问题与原问题具有相同的结构，用同原问题同样的方式求解各个子问题，然后组合这些子问题的解，以获得原问题的解的技术。

三种不同情况的递归：

- (1) 尾递归或者归纳法；
- (2) 无重叠子问题----分治
- (3) 重叠子问题，即含有冗长的调用，并允许用空间换时间的重叠子问题----动态规划

例1 阶乘函数

- 阶乘函数可递归地定义为

$$n! = \begin{cases} 1 & n = 0 \\ n(n-1)! & n > 0 \end{cases}$$

边界条件

递归方程

边界条件与递归方程是递归函数的二个要素，递归函数只有具备了这两个要素，才能在有限次计算后得出结果。

阶乘的递归算法

求n的阶乘可递归地计算如下：

factorial (n)

1. if (n = 0) then

2. return 1

3. else

4. return n*factorial(n-1)

5. end if



例2 Fibonacci数列

- 无穷数列1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ..., 被称为Fibonacci数列。它可以递归地定义为:

$$F(n) = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

边界条件

递归方程

Fibonacci数列递归算法

第n个Fibonacci数可递归地计算如下：

fibonacci (n)

1. **if** ($n \leq 1$) **then**

2. **return** 1

3. **else**

4. **return** fibonacci(n-1)+fibonacci(n-2)

5. **end if**



主要内容



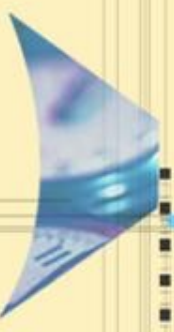
2.1 引言

2.2 归纳算法的简单例

2.3 归纳算法的设计思想

2.4. 归纳策略的经典应用





2.2 归纳算法的简单例

- 用归纳法思想导出熟知算法的派生物，以展示归纳法技术的本质。
 - Selection sort (选择排序)
 - Insertion sort (插入排序)

例3: 选择排序

基本思想

令 $A[1..n]$ 作为算法的输入,

归纳法假设: 能对 $A[1..n]$ 的后 $n-1$ 个元素, 即对 $A[2..n]$ 排序 (非降顺序)。

- 找到 $A[1..n]$ 中的最小元素, 令 $A[j]$ 是最小元素, $1 \leq j \leq n$ 。
- 如果 $j \neq 1$, 交换 $A[1]$ 和 $A[j]$ (最小值放在当前数组最前面)
- 根据归纳法假设, 能对 $A[2..n]$ 进行排序。因此通过递归调用的方式对 $A[2..n]$ 中的元素排序 (非降顺序)

算法 2.1 SELECTIONSORTREC

输入: n 个元素的数组 $A[1...n]$ 。

输出: 按非降序排列的数组 $A[1...n]$

1.sort(1)

过程 sort(i) {对 $A[i...n]$ 排序}

1. if $i < n$ then

2. $k \leftarrow i$

3. for $j \leftarrow i+1$ to n

4. if $A[j] < A[k]$ then $k \leftarrow j$ //找到 $A[i..n]$ 的最小元素

2. end for

6. if $k \neq i$ then 互换 $A[i]$ and $A[k]$

7. sort($i+1$)

8. end if

时间复杂度分析

令 $C(n)$ 表示对含 n 个元素的数组排序时元素比较总次数

- $C(1) = 0$.
- 过程sort第一次被调用时 $i = 1$ ，数组中有 n 个元素，因此在这次调用中，元素比较的总次数由两部分元素比较次数之和组成：（1） $n-1$ 次元素比较寻找最小元素；（2）递归调用sort($i+1$)时，对 $A[2..n]$ 排序需要的比较次数 $C(n-1)$ 。
- 因此，

$$C(n) = \begin{cases} 0 & \text{if } n = 1 \\ C(n-1) + (n-1) & \text{if } n \geq 2 \end{cases}$$

这个递推式的解是

$$C(n) = \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2}$$

- 由于算法的运行时间和元素的比较次数成线性关系，所以改算法的时间复杂性是 $\Theta(n^2)$ 的。

例4: 插入排序

基本思想

令 $A[1..n]$ 作为算法的输入

归纳法假设：知道如何对前 $n-1$ 个元素，也就是数组 $A[1..n-1]$ 排序（非降顺序）。

- 递归调用方式对 $A[1..n-1]$ 排序（非降顺序）
- 从后往前寻找 $A[n]$ 插入到前面排好序的数组的适当位置，设为 j ， $1 \leq j \leq n$ 。
- 这个插入位置的寻找可能导致 $A[j+1]$ ， $A[j+2]$ ， $\dots$ ， $A[n-1]$ 移到位置 $j+2$ ， $j+3$ ， $\dots$ ， n 。

算法2.2 INSERTIONSORTREC

输入: n 个元素的数组 $A[1...n]$

输出: 按非降序排列的数组 $A[1...n]$

1. $\text{sort}(n)$

流程 $\text{sort}(i)$ {对 $A[1...i]$ 排序}

1. **if** $i > 1$ **then**

2. $x \leftarrow A[i]$

3. $\text{sort}(i-1)$

4. $j \leftarrow i-1$

2. **while** $j > 0$ **and** $A[j] > x$ //如果当前值大于 x , 且不到第一个元素

6. $A[j+1] \leftarrow A[j]$ //当前值后移一位

7. $j \leftarrow j-1$

8. **end while**

9. $A[j+1] \leftarrow x$

10. **end if**

算法分析

- 显然, $C(1) = 0$.
- 过程sort的第一次调用时, $i = n$, 因此, 待排序数组有 n 个元素.
- 在最好情况时, while循环只作一次。因此, 总比较次数为递归调用sort($i-1$) 进行数组 $A[1..n-1]$ 排序的比较次数+1次while循环的元素比较.

$$C(n) = \begin{cases} 0 & \text{if } n = 1 \\ C(n-1) + 1 & \text{if } n \geq 2 \end{cases} \Rightarrow C(n) = n - 1$$

- 在最坏情况下, while循环重复 $n-1$ 次。因此, 总比较次数为递归调用sort($i-1$) 进行数组 $A[1..n-1]$ 排序的比较次数+ $(n-1)$ 次while循环的元素比较.

$$C(n) = \begin{cases} 0 & \text{if } n = 1 \\ C(n-1) + (n-1) & \text{if } n \geq 2 \end{cases} \Rightarrow C(n) = \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2}$$

主要内容



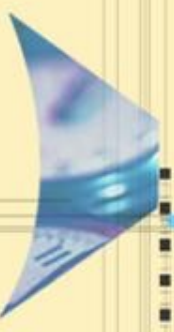
2.1 引言

2.2 归纳算法的简单例

2.3 归纳算法的设计思想

2.4. 归纳策略的经典应用





归纳算法的设计基础

- 数学的归纳证明法
 - 考虑一个带有参数 n 的问题， n 通常表示在问题的对象数。
 - 当要寻找参数为 n 的问题的解时，有时候从求解一个参数小一点的相同问题开始更容易，例如求参数是 $n-1$ ， $n/2$ 等的问题的解，然后再把解推广到包含所有 n 个对象时的问题解。
- 归纳策略的算法设计本质上就是基于众所周知的数学上的归纳证明方法。



归纳算法的一般方法

- 尾递归的思想

- 从本质上说, 给出一个带有参数 n 的问题, 用归纳法设计一个算法是基于这样一个事实: 如果我们知道如何求解带有小于 n 的参数的同样问题(它被称为归纳假设), 那么我们的任务就化为如何把解扩展到带有参数 n 的问题实例的解。

- 这种设计技术的一个优点是(一般地, 对于所有的递归算法来说都是如此): 它所设计出的算法的正确性证明已自然地嵌入了算法的描述中了。

主要内容



2.1 引言

2.2 归纳算法的简单例

2.3 归纳算法的设计思想

2.4. 归纳策略的经典应用



主要内容



2.4. 归纳策略的经典应用（选讲）

2.4.1 整数幂问题

2.4.2 多项式求值问题

2.4.3 全排列问题

2.4.4 基数排序问题

2.4.5 多数元素问题

2.4.6 汉诺塔问题

2.4.7 取棋子游戏问题



2.4.1 整数幂

问题描述

- 对求实数 x 的 n 次幂设计一个有效的算法
- 直接的方法
 - 对 x 用迭代方法自乘 n 次，这种方法是十分低效的，因为它需要 $\Theta(n)$ 乘法。按输入的大小来说，它是指数的

? Can we find much more efficient method?

一种有效的方法

- 基本思路

- 令 $m = \lfloor n/2 \rfloor$, 假设已经知道如何计算 x^m , 那么有两种情况:

- 如果 n 是偶数, 那么

$$x^n = (x^m)^2$$

- 否则,

$$x^n = x(x^m)^2$$

算法2.3 EXPREC

Input: 实数 x 和非负整数 n 。

Output: x^n .

1. $\text{power}(x, n)$

流程 $\text{power}(x, m)$ {计算 x^m }

1. **if** $m=0$ **then** $y \leftarrow 1$

2. **else**

3. $y \leftarrow \text{power}(x, \lfloor m/2 \rfloor)$

4. $y \leftarrow y^2$

2. **if** m 是奇数 **then** $y \leftarrow xy$

6. **end if**

7. **return** y



算法2.4 EXP

Input: 实数 x 和非负整数 n 。

Output: x^n .

1. $y \leftarrow 1$
2. 将 n 用二进制 $d_k d_{k-1} \dots d_0$ 表示
3. **for** $j \leftarrow k$ **downto** 0
4. $y \leftarrow y^2$
2. **if** $d_j = 1$ **then** $y \leftarrow xy$
6. **end for**
7. **return** y



算法分析

- 假定每次乘法的时间是常数，那么这两个版本的算法的运行时间是 $\Theta(\log n)$ ，它对于输入的大小来说是线性的。
- 递归版本的时间计算

$$T(n) = T(n/2) + c \quad (c=1 \text{ 或者 } 2)$$

根据主定理（情形2）， $T(n) = \Theta(\log n)$

2.4.2 多项式求值(Horner's规则)

问题描述：

假设有 $n+2$ 个实数 $a_0, a_1, \dots, a_n$ 和 x 的序列，要对如下多项式求值。

$$P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

- 直接的方法是对每一项分别求值。
- 这种方法是十分低效的，因为它需要 $n+n-1+\dots+1=n(n+1)/2$ 次乘法。

更快的方法(Horner's rule)

- 基本思想

首先观察

$$\begin{aligned} p_n(x) &= a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0 \\ &= (((\dots((a_n x + a_{n-1})x + a_{n-2})x + a_{n-3})\dots)x + a_1)x + a_0 \end{aligned}$$

这种求值的安排称为 **Horner规则**。

导出: $p_{n-1}(x) = a_n x^{n-1} + a_{n-1} x^{n-2} + \dots + a_2 x + a_1$

得到以下更有效的方法

$$p_n(x) = x p_{n-1}(x) + a_0 \quad \langle \quad p_j(x) = x p_{j-1}(x) + a_{n-j}$$

算法2.5 HORNER

Input: $n+2$ 个实数 $a_0, a_1, \dots, a_n$ x 的序列

Output: $P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$

1. $p \leftarrow a_n$
2. for $j \leftarrow 1$ to n
3. $p \leftarrow xp + a_{n-j}$
4. end for
2. return p





算法分析

- 算法 HORNER 的复杂度代价是 n 次乘法和 n 次加法,
- 这是利用归纳假设的优点得到的令人瞩目的成绩。

2.4.3 生成排列

- 问题描述：生成数 $1, 2 \dots n$ 的所有排列。
- 观察发现： 如果从 $1, 2, 3, \dots, n$ 的一个排列中删除一个整数（例如拿掉 n ），那么结果则是剩余 $n-1$ 个数（例如， $1, 2, 3, \dots, n-1$ 的一个排列。
- 归纳假设： 可以生成 $n-1$ 个数的所有排列。



第一种算法

- 基本思想

假定可以生成 $n-1$ 个数的所有排列

- 生成数 $2, 3, \dots, n$ 的所有排列，并且在每个排列的前面加上数1
- 接下来，生成数 $1, 3, 4, \dots, n$ 的所有排列，并且在每个排列的前面加上数2。
- 重复这个过程直到最后生成 $1, 2, \dots, n-1$ 的所有排列，并且在每个排列的前面加上数 n 。

算法2.6 PERMUTATIONS1

Input: 正整数 n .

Output: 数 $1, 2, \dots, n$ 的所有可能排列

1. **for** $j \leftarrow 1$ **to** n //生成数组
2. $P[j] \leftarrow j$
3. **end for**
4. $\text{perm1}(1)$

过程 $\text{perm1}(m)$

1. **if** $m=n$ **then output** $P[1 \dots n]$
2. **else**
3. **for** $j \leftarrow m$ **to** n
4. 互换 $P[j]$ and $P[m]$
5. $\text{perm1}(m+1)$
6. 互换 $P[j]$ and $P[m]$
7. **comment:** 这时 $P[m..n]=m, m+1, \dots, n$
8. **end for**
9. **end if**

算法分析

- 由于存在有 $n!$ 个排列，过程perm1的第1步共有 $nn!$ 次来
- 输出所有的排列。
- 现在计算for循环的循环次数。
 - 在第一次调用过程perm1时， $m = 1$ ，因此for循环执行 n 次加上递归调用perm1(2)的执行次数。
 - 由于当 $n=1$ 时反复次数是0，因此反复次数 $f(n)$ 能够用以下递推式表示

$$f(n) = \begin{cases} 0 & \text{if } n=1 \\ nf(n-1)+n & \text{if } n \geq 2 \end{cases}$$

算法分析（继续）

令 $n!h(n)=f(n)$ (注意 $h(1)=0$). (书本p96)

那么 $n!h(n) = n(n-1)!h(n-1) + n$

这个递推式的解是

$$h(n) = h(1) + \sum_{j=2}^n \frac{1}{(j-1)!} = \sum_{j=1}^{n-1} \frac{1}{j!} < \sum_{j=1}^{\infty} \frac{1}{j!} = (e-1)$$

这里 $e=2.7182818\dots$ 因此,

$$f(n) = n!h(n) = n! \sum_{j=1}^{n-1} \frac{1}{j!} < 2n!$$

- 所以，算法的运行时间由输出语句来决定，也就是 $\Theta(nn!)$ 。

第二种算法

- 基本思想

- 首先把 n 放入 $P[1]$ ，并且在 $P[2..n]$ 子数组中生成前 $n-1$ 个数的所有排列。
- 接下来把 n 放入 $P[2]$ ，并且在子数组 $P[1]$ 和 $P[3..n]$ 中生成前 $n-1$ 个数的所有排列。
- 然后把 n 放入 $P[3]$ ，并且在子数组 $P[1,2]$ 和 $P[4..n-1]$ 中生成前 $n-1$ 个数的所有排列。
- 这样继续下去，直到最后把 n 放到 $P[n]$ 中，并且在子数组 $P[1..n-1]$ 中生成 $n-1$ 个数的所有排列。
- 初始的时候， $P[1..n]$ 项都是自由的(没有放入生成后的排列的数)， n 个自由项设为0。

算法2.7 PERMUTATIONS2

Input: 正整数 n 。

Output: 数 $1, 2, \dots, n$ 的所有可能排列。

1. **for** $j \leftarrow 1$ **to** n
2. $P[j] \leftarrow 0$
3. **end for**
4. $\text{perm2}(n)$

过程 $\text{perm2}(m)$

1. **if** $m=0$ **then output** $P[1..n]$
2. **else**
3. **for** $j \leftarrow 1$ **to** n
4. **if** $P[j]=0$ **then**
5. $P[j] \leftarrow m$
6. $\text{perm2}(m-1)$
7. $P[j] \leftarrow 0$
8. **end if**
9. **end for**
10. **end if**

算法分析

- 由于存在有 $n!$ 个排列，过程perm2的第1步总共有 $n!$ 个排列输出。
- 现在计算for循环的迭代次数
 - for循环在每一个调用perm2(m)中被执行 n 次，再加上递归调用perm2($m-1$)中执行的次数。
 - 当过程perm2以调用perm2 (m)($m>0$)形式调用时，数组P恰好包含 m 个0，并且由于递归调用perm2 ($m-1$)将被执行恰好 m 次。因为在 $m=0$ 时，迭代次数是0，所以迭代的次数可以用递推式表示如下：

$$f(m) = \begin{cases} 0 & \text{if } m=0 \\ mf(m-1)+n & \text{if } m \geq 1 \end{cases}$$

算法分析(Continue)

令 $m!h(m)=f(m)$ (注意 $h(0)=0$). 那么,

$$m!h(m) = m(m-1)!h(m-1) + n$$

这个递推式的解是

$$h(m) = h(0) + \sum_{j=1}^m \frac{n}{j!} = n \sum_{j=1}^m \frac{1}{j!} < \sum_{j=1}^{\infty} \frac{1}{j!} < (e-1)$$

这里 $e=2.7182818\dots$

因此

$$f(m) = m!h(m) = nm! \sum_{j=1}^m \frac{1}{j!} < 2nm!$$

用 n 代入, 迭代的次数就变成

$$f(n) = nn! \sum_{j=1}^n \frac{1}{j!} < 2nn!$$

因此, 算法的运行时间是 $\Theta(nn!)$.

2.4.4 基数排序

问题描述

- 令 $L=\{a_1, a_2, \dots, a_n\}$ 是一张有 n 个数的集合，其中每个数恰好有 k 位数字，就是说每个数具有 $d_k d_{k-1} \dots d_1$ 形式，这里 d_i 是0到9中的一个数字。
- 我们研究一种排序算法对 L 进行非降序排列，使算法在几乎所有的实际应用中都以线性时间运行。
- 在这个问题中，我们对整数的大小 k 施行归纳，而不是对数 n 这个对象来用归纳法。

方法一

- 根据它们的最高位把它们分类到10个表 $L_0, L_1, \dots, L_9$ 中去，因此 $d_k = 0$ 的那些数就分到表 L_0 中， $d_k = 1$ 的那些数就分到表 L_1 中，.....。
- 这一步完成后，对于每个 L_i ($0 \leq i \leq 9$)，包含最高位是 i 的那些数。

? How to do next?

第一种选择...

- 用一种排序算法对每一张表排序，然后把排序得到的这些表连接成一张排序表。---非归纳思想
- 在最坏的情况下，所有的数字可能有相同的最高位，这就意味着它们都在1张表中，而其余的9张表是空的。
- 因此如果表内排序用的算法需要 $O(n \log n)$ 时间，则这种方法的运行时间将是 $O(n \log n)$ 的。

第二种选择...

- 对 d_{k-1} 位上每张表递归排序。--归纳思想
- 但这种方法将导致越来越多的附加表，这是很不方便的。

? Can we figure out more useful method?

惊奇的发现

- $A = \{23, 32, 34, 45, 61, 78\}$

23 **32** **34** **45** **61** **78**

61 **32** **23** **34** **45** **78**

Sort A according to the **most** significant digit

Success!

例5

- $A = \{7467, 1247, 3275, 6792, 9187, 9134, 4675, 1239\}$.

7467	6792	9134	9134	1239
1247	9134	1239	9187	1247
3275	3275	1247	1239	3275
6792	4675	7467	1247	4675
9187	7467	3275	3275	6792
9134	1247	4675	7467	7467
4675	9187	9187	4675	9134
1239	1239	6792	6792	9187

Success!

基数排序

- 基本思想:
 - 首先根据 d_1 位, 把数分到10个表 $L_0, L_1, \dots, L_9$ 中, 这样, 那些 $d_1=0$ 的数分到表 L_0 中, 那些 $d_1=1$ 的数分到表 L_1 中, 等等。
 - 接下来, 表被用 $L_0, L_1, \dots, L_9$ 的顺序接起来。
 - 再后来, 把它们按照 d_2 分到10张表中, 按顺序接起来。
 - $\circ \circ \circ \circ$
 - 最后, 把它们按 d_k 分到表中并按顺序接起来后, 所有的数就排好了序。

算法 2.8 RADIXSORT

输入：一张有 n 个数的表 $L=\{a_1 \dots a_n\}$ 和 k 位数字。

输出：按非降序排列的 L 。

1. **for** $j \leftarrow 1$ **to** k
2. 准备10个空表 $L_0, L_1, \dots, L_9$.
3. **while** L 非空
4. $a \leftarrow L$ 中的下一元素；删除 L 中的 a
2. $i \leftarrow a$ 中的第 j 位数字；将 a 加入表 L_i 中
6. **end while**
7. $L \leftarrow L_0$
8. **for** $i \leftarrow 1$ **to** 9
9. $L \leftarrow L, L_i$ {将表 L_i 加入表 L 中}
10. **end for**
11. **end for**
12. **return** L

算法分析

- 根据某一位数字把数分到表中去的方法做了k遍，每一遍的代价是 $\Theta(n)$ 时间，这样算法的时间是 $\Theta(kn)$ 。如果k是常数，运行时间就是 $\Theta(n)$ 。
- 算法用了 $\Theta(n)$ 空间，因为需要10个表，这些表全部的大小是 $\Theta(n)$ 。

基数排序算法正确性

对位数进行归纳。

- 归纳基始：算法对只有个位表示的数的排序结果显然是正确的。
- 归纳假设：假设这些数是按照它们的最低 $t-1$ 位排序是正确的
- 归纳步骤：用位 t 进行排序
 - 第 t 位相同的两个数，依据稳定性原则，保持其原始顺序不变，根据归纳假设可知，它们的排序顺序正确。
 - 第 t 位不同两个数，按他们大小的顺序排列，显然排序也是正确。

2.4.5 寻找多数元素

问题描述

- 令 $A[1..n]$ 是一个整数序列， A 中的整数 a 如果在 A 中出现的次数多于 $\lfloor n/2 \rfloor$ ，那么 a 称为多数元素。
- 例如，在序列1,3,2,3,3,4,3中，3是多数元素，因为7个元素中它出现4次。

方法1 蛮力方法（穷举）

- 基本思想

- 蛮力方法是把每个元素和其他每个元素比较，并且对每个元素计数。
- 如果某个元素的计数大于 $\lfloor n/2 \rfloor$ ，就可以断言它是多数元素；否则，在序列中就没有多数元素。
- 但是这样的比较次数是

$$n(n-1)/2 = \Theta(n^2)$$

方法2 基于排序的方法

- 基本思想

- 对这些元素排序，并且计算每个元素在序列中出现多少次。
- 最坏情况下的代价是 $\Theta(n \log n)$ 。因为在最坏情况下，排序这一步需要 $\Omega(n \log n)$ 次比较。

方法 3 基于中间元素的方法

- 基本思想

- 多数元素，在排序的序列中一定是中间元素，可以扫描这个序列来测试中间元素是否确实是多数元素。
- 寻找中间元素，就是第 $\lceil n/2 \rceil$ 元素。
- 由于中间元素可以在 $\Theta(n)$ 时间内找到（SELECT算法），这个方法要花费 $\Theta(n)$ 时间。
- 但是，中间元素寻找算法的时间函数中隐藏的常数太大，并且算法相当复杂。



Is there much more efficient method?

基于归纳的算法

观察结论 2.1

- 在原序列中去除两个不同的元素后，那么在原序列中的多数元素在新序列中还是多数元素。
- 这个观察结论支持下述寻找多数元素候选者的过程。
- 确定了候选元素后，再扫描整个数组，确定其在数组中出现次数。如果次数超过 $\lfloor n/2 \rfloor$ ，则候选元素就是多数元素，否则，数组中没有多数元素。

寻找候选元素的归纳思想

寻找候选元素的candidate过程

- 初始，令候选元素 $c=A[1]$ ，并将计数器置1；
 - 从 $A[2]$ 开始，逐个扫描元素，如果被扫描的元素和 c 相等，则计数器加1；如果元素不等于 c ，则计数器减1；
 - 如果所有的元素都已经扫描完毕并且计数器大于0，那么返回 c 作为多数元素的候选者。
 - 如果在 c 和 $A[j]$ ($1 < j < n$) 比较时计数器为0，那么对于 $A[j+1..n]$ 上的元素递归调用candidate过程。
-
- 注意减少计数器就是观察结论2.1中所述去除两个不同元素的思想的实现。

过程candidate(m)

1. $j \leftarrow m$; $c \leftarrow A[m]$; $count \leftarrow 1$

2. **while** $j < n$ **and** $count > 0$

 //当 $count == 0$ 时，循环退出，相当于除去2个不同元素。

3. $j \leftarrow j+1$

4. **if** $A[j]=c$ **then** $count \leftarrow count+1$

2. **else** $count \leftarrow count-1$

6. **end while**

7. **if** $j=n$ **then return** c //获取候选者

8. **else return** $candidate(j+1)$



算法2.9 MAJORITY

Input: n 个元素的数组 A $A[1..n]$

Output: 若存在则输出多数元素，；否则输出 none。

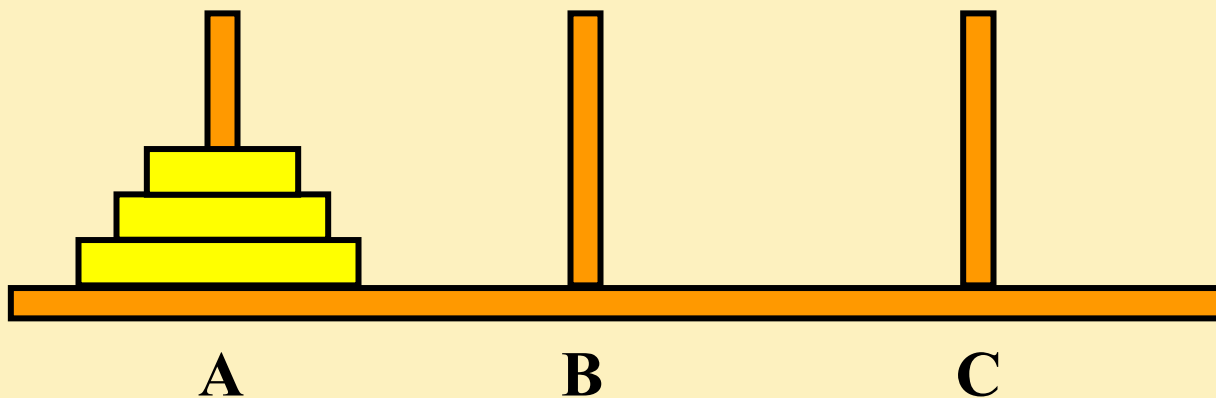
1. $c \leftarrow \text{candidate}(1)$ //获取候选者
2. $\text{count} \leftarrow 0$
3. for $j \leftarrow 1$ to n
4. if $A[j]=c$ then $\text{count} \leftarrow \text{count}+1$ //候选者进行计数
2. end for
6. if $\text{count} > \lfloor n/2 \rfloor$ then return c
7. else return none

算法分析

- candidate(1) 确定候选元素 c 只需要对 A 完成1遍扫描, 即 n 次元素比较;
- n 次 for 循环扫描 A , n 次元素比较, 对 c 的出现次数进行计数;
- 算法的时间复杂性是 $\Theta(n)$.
- 算法的空间复杂性是 $\Theta(1)$.

2.4.6 汉诺塔问题

- 存在A、B、C大小形同的3根石柱，其中A石柱从下往上按照大小顺序依次摆放着n个盘子，现在需要将A石柱的盘子全部移动到C石柱上，并且每次只能移动一个圆盘，大圆盘不能放在小圆盘上，请问该如何移动？



2.4.6 汉诺塔问题

- 当 $n=1$ 时，也就是刚开始A石柱上仅仅摆放一个圆盘，那么直接将圆盘从A石柱上移动到C石柱上即可。
- 当 $n=2$ 时，从上往下按照大小顺序将圆盘编为1号和2号，那么要将圆盘全部从石柱A移动到石柱C，首先需要将1号圆盘移动到石柱B，再将2号圆盘移动到石柱C，最后将1号圆盘移动到石柱C。
- 当 $n=3$ 时，仍然从上往下按照大小顺序将圆盘编为1号、2号和3号，此时由于问题相对复杂，所以1号和2号圆盘看做一个圆盘，即1+2号圆盘，此时需要解决的就是将1+2号圆盘和3号圆盘移动到石柱C的问题，即先将1+2号圆盘移动到石柱B，再将3号圆盘移动到石柱C，最后将1+2号圆盘移动到石柱C即可。
- 由于每次只能移动一个圆盘，那么如果要将1+2号圆盘移动到石柱B，需要将1+2号圆盘拆分为两个个体，看做将1号和2号圆盘移动到石柱B，同理将1+2号圆盘移动到石柱C。



2.4.6 汉诺塔问题

- 当 $n=n$ 时，将圆盘自上向下编为1号、2号、3号..... n 号，同理将1号到 $n-1$ 号圆盘看做一个圆盘，即 $\sum(n-1)$ 号圆盘，此时解决的就是将 $\sum(n-1)$ 号圆盘和 n 号圆盘移动到石柱C的问题，即先将 $\sum(n-1)$ 号圆盘移动到石柱B，再将 n 号圆盘移动到石柱C，最后将 $\sum(n-1)$ 号圆盘移动到石柱C。因为将第 n 号圆盘移动到石柱C后，无论前 $n-1$ 个圆盘怎么移动，都不需要再次移动第 n 号圆盘，即父问题与子问题相对独立且互不影响，因此可以将 $\sum(n-1)$ 号圆盘的问题同理向下拆分移动。

2.4.6 汉诺塔问题

Hanoi(A, C, n) // 将 A 的 n 个盘子按要求移到 C

1. **if** $n=1$ **then** move (A, C) // 将 A 的1个盘子移到 C

2. **else**

 Hanoi($A, B, n-1$)

3. move(A, C)

4. Hanoi($B, C, n-1$)

5. **end if**

$$T(n) = 2 T(n-1) + 1, \quad T(1) = 1$$



2.4.7 棋子移动游戏

– 问题描述：有 $2n$ ($n \geq 4$ 为整数) 棋子，黑白棋子各 n 个，初始状态为：

○○○○●●●●__

右边两空位

– 终止状态为 __○●○●○●○●

左边两空位

– 将棋子从初始状态移成终止状态。移动规则如下：

- 1) 每次必须同时移动相邻两个棋子；
- 2) 移动方向不限；
- 3) 每次移动必须跳过若干棋子。

https://blog.csdn.net/qq_45815776



对于四对棋子的情形，4，5移动到9，10；8，9移动到4，5；
2，3移动到8，9；7，8移动到2，3；1，2移动到7，8

- 00001111__ -> move(4,9)
- 000__11101 ->. move (8,4)
- 0001011 __1 ->. move(2,8)
- 0 __1011001 ->. move(7,2)
- 010101 __01 -> move(1,7)
- __01010101 -> 求解



- 
- 如果是五对棋子，移动中间两个一白一黑到最后位置，再从后方移动两个黑色到刚刚的空位，就会发现四个以上都是这个套路，最终都会变成解四个。

- 0000011111__ -> move(5,11); $n \rightarrow 2n+1$

- 0000__111101 -> move(9,5); $2n-1 \rightarrow n$

- 00001111__01

- 如果是六对棋子，移动中间两个一白一黑到最后位置，再从后方移动两个黑色到刚刚的空位，变为五对棋子的情形。
。。。四个以上都是这个套路，最终都会变成解四个。
- 000000111111__ ->. move(6, 13); n,->2n+1
- 00000__1111101 ->. move(11, 6); 2n-1-> n
- 0000011111__01 -> 变成5对棋子(n-1对)的情形;
move(5,11);
- 0000__11110101 -> move(9,5);
- 00001111__0101 -> 变成4对棋子情形



Algorithm MoveChess

Input: $X[1..2n+2]$, $X[1]=X[2]=\dots=X[n]=0$ 代表 n 个白棋; $X[n+1]=X[n+2]=\dots=X[2n]=1$ 代表 n 个黑棋; $X[2n+1]=X[2n+2]=-1$, 代表两个空位;

Output: 变换后的 $X[1..2n+2]$, $X[1]=X[2]=-1$ 代表空位; $X[3]=X[5]=\dots=X[2n+1]=0$ 代表白棋; $X[4]=X[6]=\dots=X[2n+2]=1$ 代表黑棋

chessmove (n); 将 n 对黑白棋子从初始移动到目标状态

Procedure chessmove(n)

if $n=4$ then

 move(4,9)

 move (8,4)

 move(2,8)

 move(7,2)

 move(1,7)

else

 move(n , $2n+1$);

 move($2n-1$, n)

 chessmove($n-1$)

end if

Procedure move(i, j)

 exchange ($X[i], X[j]$)

 exchange ($X[i+1], X[j+1]$)

设 n 对棋子从初始移动到目标状态的时间 $T(n)$

$$T(n) = \begin{cases} b. & n = 4 \\ T(n-1) + c. & n > 4 \end{cases}$$